

Markdown Handout Builder

Dialects 版完全指南 · 功能与配置全参考

Markdown Handout Builder

2026.07.11

2.0-dialect

前言

这是 **Markdown Handout Builder (mhb) Dialects** 版的完全指南，覆盖本版次的**全部功能与全部配置项**：从命令行流水线、`structure:` 文档结构语言、layouts / parts / includes、每章页眉页脚策略，到 Obsidian 方言的完整语法支持、主题系统与官方 PDF 打印管线。

版次与对应关系

对象	版次	说明
npm 包 <code>markdown-handout-builder</code>	<code>2.2.0-beta.1</code>	dialects 分支预发布， <code>next</code> dist-tag
本指南（文档版次）	<code>2.0-dialect</code>	与 Obsidian 语法 Showcase 同版次
使用手册（原版功能）	<code>2.0</code>	<code>docs/handbook/</code> ，英文

如何阅读

- **第一次使用**：读第一部分（总览与命令行），跑通 `check → build → pdf`。
- **要排一本书**：读第二部分（文档结构语言），它是本版次的核心能力。
- **从 Obsidian vault 出版**：读第三部分，配合成品《Obsidian Markdown · 静态语法全覆盖》对照验证。
- **查配置**：直接翻最后一章《book.yml 全键参考》，全部键、类型、默认值一页速查。

本书自身就是用它所讲解的能力排版的：`structure:` 长写法、layouts 继承、五个满版出血的篇章隔页、流内目录，以及参考章的独立页眉策略。仓库里的 `[docs/dialect-guide/book.yml]` 即为可运行的完整示例。

说明

本指南描述 dialects 版行为。凡与原版（2.0）不同之处，正文中都会标注「dialects 版新增」。

目录

第一部分 · 总览	3
01 · 工具总览	4
02 · 命令行与流水线	7
第二部分 · 文档结构语言	10
03 · 文档结构语言	11
04 · Layouts、分页与导航	15
05 · 每章页眉页脚策略（running）	18
第三部分 · Obsidian 方言	21
06 · Obsidian 方言：启用与解析模型	22
07 · Obsidian 语法全参考	24
第四部分 · 外观与 PDF	28
08 · 主题、样式与封面	29
09 · 官方 PDF 管线	33
第五部分 · 参考	37
book.yml 全键参考	38

第一部分·总览

定位、安装与命令行流水线

01 · 工具总览

本章内容

mhb 是什么	4
设计取向	4
dialects 版新增能力一览	5
安装	5
仓库约定	6
一条最小配置	6

mhb 是什么

Markdown Handout Builder 把一组纯 Markdown 笔记构建成**两种成品**：

1. **自包含 HTML**（`dist/handout.html`）——阅读用，内联全部 CSS 与 KaTeX 字体引用，离线可用；
2. **官方 PDF**（`dist/handout.pdf`）——打印/分发用，由 Playwright Chromium 从同一份 HTML 打印生成，再经 pdf-lib / pdfjs 后处理（页码、书签、元数据、覆盖层）。

核心原则：**每份 PDF 只从对应的同一份 HTML 打印生成**，不存在第二套模板；HTML 与 PDF 的差异只来自打印介质与官方管线的后处理。

设计取向

- **内容与策略分离**。Markdown 里只有内容；顺序、分页、页眉页脚、目录归属全部在 `book.yml` 声明。排版策略（layouts）不是第二套样式语言——外观仍然交给 CSS。
- **不自动编号**。章节号、定理号、图号、公式号都由作者写在内容里（标题、环境名、图注、KaTeX `\tag{}`），所见即所得。
- **错误必须显式**。拼错的键、指错的路径、成环的继承在 `mhb check` 阶段就报错，绝不静默变成空白页。
- **默认严格、可移植**。默认方言是严格 Markdown（CommonMark + 常用扩展，禁 raw HTML）；Obsidian 方言按项目显式启用。

dialects 版新增能力一览

能力	一句话	详见
<code>structure</code> : 结构语言	<code>chapters</code> : 的书籍级替代: 显式 <code>type</code> : 、部 (part) 、包含 (include)	第 03 章
命名 layouts	可继承、可复用的条目默认值包	第 04 章
flow / navigation	分页与目录/书签归属逐条目独立控制	第 04 章
每章 running 策略	单章关闭或改写页眉/页脚槽位与样式	第 05 章
声明式特殊页	divider (篇章隔页, 支持满版出血)、blank (空白页)、contents (目录布点)	第 03 章
Obsidian 方言	wikilink、transclusion、properties、callouts、Mermaid..... 全静态语法	第 06–07 章
<code>mhb inspect</code>	打印归一化后的扁平文档流 (<code>--json</code> 可机读)	第 02 章
<code>book.schema.json</code>	随包分发的 JSON Schema, 编辑器校验与补全	第 02 章

安装

要求: Node.js ≥ 20 ; 渲染 PDF 需要 Playwright Chromium。

在笔记仓库中:

```
npm install -D markdown-handout-builder
npx mhb install-browser          # 安装本包锁定的 Chromium
```

新仓库可以直接生成脚手架:

```
npx markdown-handout-builder init
```

`init` 生成 `book.yml` (带 schema 头注释)、`notes/` 示例章节、`package.json` 脚本、`.gitignore`、GitHub Actions 工作流与 `WRITING_RULES.md`, 已存在的文件会跳过 (`--force` 才覆盖) 。

仓库约定

```
book.yml          # 唯一配置（或用 --config 指定别处）
notes/            # 正式内容；本地图片放 notes/assets/
  00-intro.md
  assets/
dist/             # 构建产物（HTML、PDF、index.html、复制的资源）
```

- 章节文件通常有且只有一个 `#` 一级标题：它是章名，进入主目录与 PDF 书签，也是每章 running 策略的页区间锚点。
- `notes/` 下未被结构收录且未被 transclusion 引用的 `.md` 会得到警告；文件名以 `_` 开头视为草稿，不提醒。
- `notes/assets/` 下未被引用的文件同样会被警告。

一条最小配置

```
title: "My Handout"
language: "zh-CN"
chapters:
  - notes/00-intro.md
  - notes/01-topic.md
output:
  html: dist/handout.html
  pdf: dist/handout.pdf
```

除 `title`、`chapters` / `structure` 二选一、`output` 外，其余键全部可选。完整注释模板见包内 `book.example.yml`，全键速查见本书第 10 章。

02 · 命令行与流水线

本章内容

命令总表	7
check: 构建前把关	7
inspect: 先看展平结果再渲染	8
build: 唯一 HTML 渲染源	8
pdf: 官方打印管线	9
serve: 本地预览	9
编辑器校验: book.schema.json	9
典型 CI	9

所有命令均可用 `mhb <命令>` 或 `npx markdown-handout-builder <命令>` 调用; `mdhb` 是等价别名。默认读取当前目录的 `book.yml`, 任何命令都接受 `--config <file>` 指向别处。

命令总表

命令	作用
<code>mhb init</code>	生成最小笔记仓库脚手架 (<code>--force</code> 覆盖已存在文件)
<code>mhb check</code>	校验配置、文档结构、本地资源与方言语法/链接
<code>mhb inspect</code>	打印归一化后的文档结构 (<code>--json</code> 输出机读 JSON)
<code>mhb build</code>	渲染 <code>dist/handout.html</code> 、主题变体与 <code>dist/index.html</code>
<code>mhb pdf</code>	从已生成的 HTML 打印官方 PDF (<code>render-pdf</code> 同义)
<code>mhb serve</code>	本地预览 + 保存即重建 HTML (<code>--port</code> 指定端口, 默认 8000)
<code>mhb all</code>	依次执行 check、build、pdf
<code>mhb install-browser</code>	安装本包锁定版本的 Playwright Chromium
<code>mhb install-deps</code>	在 CI (Linux) 安装 Chromium 系统依赖

推荐把它们映射为 npm scripts (`init` 已代劳) : `npm run check / build / pdf / serve / all`。

check: 构建前把关

`mhb check` 汇总报告全部错误 (`x`) 与警告 (`⚠`), 任一错误即退出码 1:

- `book.yml` 存在、可解析、顶层是映射；
- `structure` / `chapters` 归一化成功：文件存在、扩展名可识别、`layout / part / include` 合法、`flow / navigation / running` 策略约束满足；
- 章节内本地图片存在（相对章节文件解析）；
- 标准方言拒绝 Obsidian 专有语法（围栏与行内代码除外）；Obsidian 方言则解析 `vault`，逐一验证 `wikilink / embed` 目标与片段，递归检查被 `transclusion` 的笔记，报告歧义目标；
- `toc / chapter_toc / pdf / themes / labels` / 封面封底组件 / `custom_css` 路径等配置类型校验；
- 警告：未收录的 `notes/*.md`、未引用的 `notes/assets/*`、未知占位符、被忽略的组合（如流内目录 + `count_toc: false`）。

inspect：先看展平结果再渲染

`structure` 支持部嵌套与递归包含，`mhb inspect` 打印展平后的最终序列与每条目继承解析后的策略：

```
Document structure (13 entries, source: structure)
#   TYPE      ROLE    LAYOUT    BEFORE  AFTER  TOC          OUTLINE  RUNNING
SOURCE
1   insert    insert   -         page    auto   no           yes      +H/+F
notes/00-preface.md
2   contents  contents -         -        -        -           yes      +H/+F
3   divider  part     -         page    auto   第一部分 · 总览 yes      +H/+F  第
一部分 · 总览
4   chapter  chapter  body      page    auto   yes          yes      +H/+F
notes/01-overview.md
...
13  chapter  chapter  reference page    auto   book.yml 全键参考 yes      *H/+F
notes/10-config-reference.md
```

RUNNING 列的记号：`+H/+F` 继承全局页眉/页脚；`-H / -F` 该章关闭；`*H / *F` 槽位被改写；末尾 `/S` 表示 style 有覆盖。`--json` 输出同等信息的 JSON，适合脚本消费。

build：唯一 HTML 渲染源

`mhb build` 按归一化文档流渲染 Markdown，产出：

- `dist/handout.html` —— 默认主题成品（`print.css` 与 `KaTeX CSS` 内联）；
- `dist/handout.<theme>.html` —— 每个非默认主题一份变体；
- `dist/index.html` —— 落地页（含各主题入口与章节直达）；
- 复制 `notes/assets/` → `dist/assets/`、`KaTeX` 字体 → `dist/assets/katex-fonts/`、`THIRD_PARTY_NOTICES.md`；Obsidian 方言另复制被引用的 `vault` 附件 → `dist/vault/`，用到 Mermaid 时复制 `mermaid.min.js`。

pdf: 官方打印管线

`mhb pdf` 打开各主题 HTML，切换打印介质渲染 PDF。页码回填、计页剔除、封面/封底/出血覆盖、每章 running 策略、元数据与书签的完整机制见第 09 章。

serve: 本地预览

零依赖静态服务器托管 `dist/`，并监听：`book.yml`、`notes/`、本地模板、外部 structure 文件与**全部递归 include 文件**（构建成功后自动刷新监听清单）。保存即重建 HTML 并自动刷新浏览器；PDF 刻意保持手动（较慢）。

```
npm run serve -- --port 8000
```

编辑器校验: book.schema.json

包内随附 `book.schema.json`。在 `book.yml` 首行加一条注释，即可在 VS Code（YAML 扩展）等编辑器中获得键名补全与类型校验：

```
# yaml-language-server: $schema=./node_modules/markdown-handout-builder/book.schema.json
```

`mhb init` 生成的配置已带此头。

典型 CI

`init` 生成的工作流：装依赖 → `mhb install-browser -- --with-deps chromium` → 装 CJK 字体 → `check + build + pdf` → 上传产物 / 部署 Pages。要点：CI 里 PDF 渲染完全可行，Chromium 建议按 Playwright 版本缓存。

第二部分 · 文档结构语言

structure / chapters · layouts · parts · includes

03 · 文档结构语言

本章内容

条目的三种写法	11
全部条目类型	12
chapter —— 渲染章节	12
insert —— 特殊页	12
divider —— 篇章隔页（无文件）	12
blank —— 有意的空白页	13
contents —— 主目录布点	13
part —— 部（语义分组）	13
include —— 递归包含	14
继承与合并顺序	14
结构级校验	14

文档顺序由 `book.yml` 里的 **chapters:**（紧凑形）或 **structure:**（语义形）声明。两者二选一，同时出现是错误；它们归一化为同一套扁平渲染序列，因此紧凑形能做的，语义形都能做，反之亦然。整份清单也可以外置：`chapters: chapters.yml`（或 `structure: structure.yml`），指向一个 YAML 列表文件。

条目的三种写法

① 字符串——就是文件路径，扩展名决定角色：

```
chapters:
- notes/00-intro.md      # .md/.markdown → chapter (渲染章节)
- notes/interlude.html   # .html/.htm   → insert (受信原样插页)
```

② 映射（紧凑形）—— `path` 加逐条目选项：

```

- path: notes/02-deep.md
  as: insert          # 角色覆盖 (role 同义) ; .html 不可作 chapter
  class: deep-dive    # 附加到 <section> 的 CSS class
  layout: body        # 引用命名 layout (见第 04 章)
  chapter_toc: true    # 开章小目录 (仅章节有意义)
  toc: "第二章 (改名)" # 主目录行文案; toc: false 则整章不进主目录
  navigation: { level: 2 } # 见第 04 章
  flow: { break_before: page, break_after: auto }
  running: { footer: false } # 见第 05 章

```

③ 显式 **type:** (语义形) ——同样的键，判别子写在条目上：

```

structure:
- type: chapter      # chapter | insert | divider | blank | contents | part |
  include            #
  path: notes/00-intro.md

```

映射条目必须恰有一个类型判别 (`path` / `divider` / `blank` / `contents` / `part` / `include` 之一，或显式 `type:`)；未知键一律报错——拼写错误不可能静默变成空白页。

全部条目类型

chapter —— 渲染章节

Markdown 渲染；一级标题成为章名（主目录行 + PDF 书签 + running 锚点）。可选键：`class` `layout` `chapter_toc` `toc` `navigation` `flow` `running`。

insert —— 特殊页

两种来源，同一角色（不进主目录、按插页排版、带运行页眉）：

- **.html 文件**：受信原样片段，`{{title}}` 等全局占位符会被填充。是 Markdown 表达不了的版式的逃生门。
- **.md + `as: insert` (或 `type: insert`)**：前言、致谢、版权页——照常渲染 Markdown，但不进主目录。注意：raw HTML 插页不能携带 running 策略（策略需要 Markdown 一级标题作页区间锚点）。

divider —— 篇章隔页（无文件）

一整页的隔页卡，全部键：

```

- divider:                                # 或 type: divider 平铺同名键
  title: "第一部分"                        # 真实 <h1>: 进书签; bleed 时必填
  subtitle: "Foundations"
  note: "可选的第三行说明"
  class: part-one                         # 交给 custom_css 深度定制
  layout: quiet                           # 也可引用 layout
  background: "linear-gradient(150deg, #1d2a44, #5f58b6)" # 任意 CSS 颜色/渐变
  color: "#ffffff"
  bleed: true                             # 官方 PDF 满版出血 (独立单页打印 + 整页覆盖)
  toc: "第一部分"                         # 主目录行 (divider 默认不进; part 默认进)
  navigation: { level: 1 }
  flow: { break_before: page }

```

隔页恰占一页由打印规则保证（与封底同一套「安全 min-height」模式）。`bleed: true` 的背景在官方 PDF 中铺满整页；浏览器直接打印时铺满正文区（内容盒）。至少要有 `title / subtitle / note / class` 之一。

blank —— 有意的空白页

```

- blank: true                             # 一页
- blank: 2                                # 1-20 页
- type: blank                             # 长写法可带 count / class / layout / flow
  count: 1

```

用于双面印刷的对页排版。空白页照常计页、带页眉页脚，但不能携带 running 策略（没有可寻址的标题锚点）。

contents —— 主目录布点

```

- contents: true                           # 或 type: contents; 全书至多一次

```

不写此条目时，主目录默认在封面之后。写了它，目录就渲染在文档流中的这个位置——经典的前置页顺序（扉页 → 前言 → 目录 → 正文）由此实现。**流内目录恒计页**：`pdf.page_numbers.count_toc: false` 与之冲突时被忽略并警告（页码拼接机制假设被剔除的目录紧随封面）。

part —— 部（语义分组）

part = 一页 divider + 一组子条目 + 可继承的 defaults:

```

- type: part                                # 紧凑形为 - part: {...}
  title: "第一部分"
  subtitle: "Foundations"
  bleed: true
  navigation: { label: "I · Foundations", level: 1 }
  defaults:                                # 后代条目的默认值（可被子条目覆盖）
    layout: body
    class: part-one-page
  children:                                # chapters / structure 是同义键
    - notes/01-a.md
    - include: parts/more.yml
    - type: part                            # 部可以再嵌套
      title: "第一部分 · 补编"
      children: [notes/01-c.md]

```

部标题拥有真实标题与书签；子章节的主目录层级自动加深一层（部在 level 1，则子章 level 2），也可用 `defaults.navigation.level` 显式指定。divider 的全部键（背景、出血、note.....）对 part 同样可用；区别是 part 的 `toc` 默认开启（divider 默认关闭）。展平发生在归一化阶段，渲染管线始终消费线性序列。

include —— 递归包含

```

- include: parts/foundations.yml           # 或 type: include + path:

```

被包含文件是一个 YAML 列表（或带 `structure:` / `chapters:` 键的映射），内容可再嵌 part / include。**include** 路径相对于包含它的 YAML 文件解析；其中的章节路径仍相对于 `book.yml`。包含环是硬错误，`mhb serve` 会自动监听全部递归包含文件。

继承与合并顺序

每个条目的最终策略 = **layout**（含 **extends** 链）→ 外层 **part** 的 **defaults**（逐层）→ 条目自身，后者覆盖前者；`class` 取并集，`running` 的 header / footer / style 深合并（见第 05 章）。用 `mhb inspect` 验证展平与继承结果。

结构级校验

- `structure` 与 `chapters` 并用 → 错误；
- 至少一条 `.md` / `.html` 文件页；`contents` 至多一次；
- 未知键 / 类型键并存 / 空 part / include 环 / layout 环或未知名 → 错误；
- 携带 `running` 策略的条目不能与后一条目共享物理页（要求自身 `break_before: page`，且不被后一条 `break_before: auto` 贴上来）。

04 · Layouts、分页与导航

本章内容

命名 layouts	15
flow —— 分页流	16
navigation —— 目录与书签归属	16
主目录与每章小目录	17

命名 layouts

layouts: 定义可复用的条目默认值包。可用键与条目/ defaults 一致: extends、class、chapter_toc、toc、navigation、flow、running。

```
layouts:
  body:
    class: layout-body
    chapter_toc: true

  compact:
    extends: body           # 单继承; 解析在渲染前完成
    class: layout-compact  # class 与父层取并集
    flow:
      break_before: auto    # 其余键逐字段覆盖

  no-footer:
    extends: body
    running:
      footer: false         # running 深合并 (详见第 05 章)

  structure:
    - type: chapter
      path: notes/short-note.md
      layout: compact
```

规则与校验:

- layout 名须匹配 `[A-Za-z_][A-Za-z0-9_-]*`;
- extends 引用未知名、继承成环 → 硬错误 (check 报出完整环路径);
- 条目引用未知 layout → 硬错误;

- 合并语义：`class` 并集去重；`chapter_toc`、`flow.*`、`navigation.*` 逐字段后者覆盖前者；`running` 的 header / footer / style 深合并。

layout 管策略与钩子，不管字体字号——外观交给 CSS。构建产物给每个条目留了两个稳定钩子：生成的 class（`class:` 并集）与 `data-layout="<名>"` 属性，配合 `style.custom_css` 使用：

```
.chapter[data-layout="compact"] h2 { margin-top: 1em; }
.layout-body .chapter-toc { border-left-color: #5f58b6; }
```

flow —— 分页流

每个条目独立声明分页，取值均为 `page` | `auto`：

```
- type: chapter
  path: notes/appendix.md
  flow:
    break_before: page      # 默认：每个条目起新页
    break_after: auto       # 默认：不强制结束页
```

- `break_before: auto` 让本条目紧跟上一条目排版（如一组短插页连排）；
- `break_after: page` 强制本条目结束后翻页；
- 首个正文条目自动豁免起页断（紧随封面/目录，不出空页）；
- 携带 `running` 策略的条目必须 `break_before: page`，且其后条目不得以 `break_before: auto` 贴上来——一张物理页只能有一套页眉策略，`check` 会拦下违例组合；
- `break-before: recto/verso`（奇偶页起章）刻意未提供：当前 Chromium 打印忽略该属性，宁缺毋滥。需要右页起章时用 `blank:` 手工垫页。

navigation —— 目录与书签归属

导航与分页正交，四个键：

```
- type: chapter
  path: notes/appendix.md
  navigation:
    toc: true                # 进主目录 (false = 不进；锚点与书签保留)
    label: "附录 A"         # 主目录行文案（覆盖一级标题文本）
    level: 2                # 主目录层级 1-6
    outline: true            # 是否进入 PDF 书签
```

- 紧凑形的 `toc: false` / `toc: "文案"` 是 `navigation.toc` / `navigation.label` 的简写；

- `level` 独立于 Markdown 标题深度：条目 `level: 2` 时，其章内 `h2` 在主目录中呈现为第 3 层，依此类推（part 的子条目自动获得「部层级 + 1」为基准）；
- 约束：`outline: false` 目前要求同时 `toc: false` ——真实目录页码靠 PDF 书签目的地映射，标题不进书签就无法回填页码；
- running 策略要求 `outline: true`（页区间映射同样依赖书签）。

主目录与每章小目录

全局配置（两者的行都带真实 PDF 页码回填）：

```
toc:
  enabled: true           # false = 不生成主目录
  title: "目录"
  depth: 2               # 收录层级 1-3

chapter_toc:
  default: false         # true = 每章默认开小目录
  title: "本章内容"      # "" 隐藏小标题
  depth: 3               # 收录 h2..h<depth>
  class: my-chapter-toc  # 附加 class
```

每章小目录列出本章 `h2..depth` 级标题、插在一级标题之后，位于正文流内（恒计页）。条目/layout 的 `chapter_toc: true|false` 覆盖全局默认。

05 · 每章页眉页脚策略（running）

本章内容

写法	18
占位符	19
前提条件（check 会逐条把关）	19
实现机制（为什么这样约束）	19
常见配方	20

dialects 版允许单个条目改写官方 PDF 的页眉/页脚——关闭某条带、替换某个槽位、或逐字段覆盖排版样式——而不影响其余页面与全局配置。本章即为活演示：最后一章《book.yml 全键参考》通过 `reference layout` 把页眉中位换成了章名。

写法

`running` 接受 `false` 或一个映射：

```
- type: chapter
  path: notes/special.md
  running: false          # 本章页眉页脚全关

- type: chapter
  path: notes/special.md
  running:
    footer: false        # 只关页脚，页眉保留

- type: chapter
  path: notes/special.md
  running:
    header:
      center: "{{chapterTitle}}"  # 只改中位；left/right 继承全局 pdf.header
    footer:
      left: "Internal draft"
      center: "{{page}} / {{total}}"
      right: "Chapter A"
    style:                  # 逐字段覆盖全局 pdf.header_footer_style
      font_size: "8px"
      color: "#667085"
```

- `header / footer` : `true` (继承全局)、`false` (关闭该条带)、或 `left / center / right` 槽位映射 (未写的槽位继承全局值) ;
- `style` : `font_size / color / font_family / offset` 四键, 逐字段继承全局;
- 策略可放进 `layout` 或 `part` 的 `defaults` 复用, **深合并**: 子层只写 `header.center` 时, 父层的 `footer` 与 `style` 原样保留;
- 全局 `pdf.header_footer: false` 时, 显式配置了 `header / footer` 的章只把该条带 **opt-in** 回来, 未配置的条带保持关闭。

占位符

槽位内容支持全部全局占位符 (见第 10 章总表), 外加两个章节局部量:

占位符	值
<code>{{chapterTitle}}</code>	本条目的一级标题文本
<code>{{sectionTitle}}</code>	同上 (同义别名)

`{{page}}` / `{{total}}` 在策略页上同样是**逻辑页码**——封面、目录、封底被 `count_*: false` 剔除时, 数值与全书其余页完全一致。

前提条件 (check 会逐条把关)

1. 条目必须 `flow.break_before: page` (默认即是), 且后一条目不得 `break_before: auto` 贴上来——一张物理页只能有一套页眉策略;
2. 条目必须是 **Markdown 页**且含一级标题: 该标题的 PDF 书签目的地是页区间映射的锚点 (raw HTML 插页与 blank 页不可携带策略; 无标题的 Markdown 页构建时警告并回退全局页眉);
3. `navigation.outline: true` (默认) ——不进书签就无法定位物理页区间。

实现机制 (为什么这样约束)

Chromium 打印只支持**一套全局**页眉/页脚模板, 无法按页切换。官方管线因此在最终 PDF 上做后处理:

1. 用条目一级标题的书签目的地, 把该章映射为**物理页区间** (起页 = 本章标题所在页, 止页 = 下一策略章起页前一页, 末章止于封底之前);
2. **关闭** (`false`): 把对应页边距条带用页面基底色重新涂平——正文从条带之外开始, 内容、链接笔记、书签一概不动;
3. **改写** (槽位/样式): 把新页眉作为单页 PDF 由 Chromium 渲染 (保留浏览器字形整形与 CJK 排版), 裁出条带贴到目标页——每页独立渲染, 逻辑页码逐页正确;
4. 封面与封底不在任何策略区间内 (它们本就有独立的覆盖机制)。

常见配方

```
layouts:
  no-footer:                # 图版章：无页脚
    running: { footer: false }
  chapter-header:          # 书籍式：页眉中位显示章名
    running:
      header: { center: "{{chapterTitle}}" }

structure:
- type: part
  title: "正文"
  defaults: { layout: chapter-header } # 整部继承
  children:
    - notes/01.md
    - notes/02.md
- type: chapter
  path: notes/gallery.md
  layout: no-footer
```

提示

用 `mhb inspect` 的 RUNNING 列（`+H/+F`、`-F`、`*H`、`/S`）快速核对每章最终生效的策略。

第三部分 · Obsidian 方言

vault 感知渲染与全语法参考

06 · Obsidian 方言：启用与解析模型

本章内容

启用	22
vault 索引	22
链接解析优先级	23
目标去向	23
check 在方言下的深度校验	23
与结构语言的关系	23

Obsidian 方言让 mhb 直接从一个 Obsidian vault 出版：wikilink、transclusion、properties、callouts、标签、Mermaid 与官方附件类型全部静态渲染进 HTML 与 PDF。**默认不启用**——默认方言保持严格、可移植的 Markdown。

启用

```
markdown:
  dialect: obsidian          # "standard" (默认) 或 "obsidian"
  obsidian:
    vault_root: "."         # vault 根目录, 相对 book.yml; 默认 "."
    properties: visible     # frontmatter 展示: visible | hidden | source
```

- `properties: visible` ——每章顶部渲染 Obsidian 风格的属性表；`hidden` ——解析但不展示；`source` ——以 YAML 源码块展示。
- **信任模型**：Obsidian 模式与 Obsidian 本体一致，放行 raw HTML。只对受信内容启用本方言。
- 标准方言下，正文出现 `[[wikilink]]` / `![[embed]]`（围栏与行内代码之外）会被 `check` 拒绝并提示改写为标准语法。

vault 索引

启用后，`vault_root` 被一次性扫描建立索引：

- **收录扩展名**：`.md`、`.base`、`.canvas`、`.pdf`，图片（avif/bmp/gif/jpeg/jpg/png/svg/webp）、音频（flac/m4a/mp3/ogg/wav/webm/3gp）、视频（mkv/mov/mp4/ogv/webm）；
- **跳过**：`.git`、`.obsidian`、`node_modules`、`dist` 及一切点开头目录；
- 每个 `.md` 的 frontmatter 被解析：`aliases`（含逗号分隔标量形）进入别名索引，`cssclasses` 应用到章节 `<section>`，坏 YAML 在 `check` 报错（章节）或警告（vault 其他笔记），构建时降级为无属性渲染。

链接解析优先级

[[目标]] 按以下顺序解析，全部大小写不敏感、Unicode NFKC 归一：

1. **精确 vault 路径**（含或不含 `.md` 后缀）；
2. **相对当前笔记的路径**；
3. **文件名 / 别名**：候选按目录距离（与当前笔记的最近公共祖先）排序，最近者胜；**等距歧义**取字典序最先者，并在 `check` 与构建时给出警告（写明选中了谁）。

解析失败 → `check` 报错（含行号）；构建仍会完成，未解析链接渲染为带下划线点线的 `unresolved` 样式并告警。

目标去向

- 链到**已收录章节**的笔记 → 文内锚点（`#章节标题`），跨章前向链接在全部章节渲染完成后统一回填；
- 链到**未收录**的 vault 笔记/附件 → 文件复制到 `dist/vault/<相对路径>` 并按源文件链接；
- 标题片段 `[[笔记#标题]]`、多级 `[[笔记#H2#H3]]`（按层级后缀匹配，优先正文本体而非 transclusion 副本）、块片段 `[[笔记#^block-id]]` 都在**构建前验证存在性**。

check 在方言下的深度校验

- 每条 wikilink / embed 的目标与片段存在性（含表格里 `\|` 转义形）；
- 递归进入被 transclusion 的笔记继续校验（环安全）；
- 歧义目标警告；`vault_root` 不存在、`properties` 非法取值等配置错误；
- 未收录且未被嵌入的 `notes/*.md` 警告（`_` 前缀豁免）。

与结构语言的关系

方言与第 03–05 章的结构能力完全正交：Obsidian 书同样可用 `structure:`、`layouts`、`divider`、`running` 策略——本仓库的《Obsidian 语法 Showcase》即混用两者（方言 + 出血隔页）。

07 · Obsidian 语法全参考

本章内容

内部链接 (wikilinks)	24
嵌入 (transclusion 与附件)	25
Properties (frontmatter)	25
标签	26
注释	26
Callouts	26
任务列表	26
Mermaid	26
其余共享语法	27
静态边界 (刻意不做的)	27

本章逐项列出方言模式支持的全部静态语法。可运行的对照成品见《Obsidian Markdown · 静态语法全覆盖》（`docs/obsidian-showcase/`，输出 `dist/showcase/`）。

内部链接 (wikilinks)

写法	效果
<code>[[Note]]</code>	按第 06 章优先级解析；章内笔记 → 锚点，vault 文件 → 复制并链接
<code>[[Note 显示文本]]</code>	自定义链接文本（表格单元格内写 <code>\ </code> ）
<code>[[Note#Heading]]</code>	标题片段；构建前验证存在
<code>[[Note#H2#H3]]</code>	多级标题路径，按 层级后缀 匹配；优先章节正文而非嵌入副本
<code>[[Note#^block-id]]</code>	块引用片段
<code>[[#Local Heading]]</code>	同页标题
<code>[Markdown 形]</code> <code>(Note.md#Heading)</code>	标准 Markdown 链接同样走 vault 解析（URL 编码可用）
<code>obsidian://...</code>	协议链接原样保留

块标识符：段落行尾 `^id`（阅读视图与 PDF 中隐藏）；列表/整块可用**独立成行**的 `^id` 标注上方相邻块。

嵌入（transclusion 与附件）

写法	效果
<code>![[Note]]</code>	整篇嵌入：递归渲染（含其内部嵌入），环引用被拦截并警告
<code>![[Note#Heading]]</code>	只嵌入该标题区段（到同级下一标题为止）
<code>![[Note#^block-id]]</code>	只嵌入该块（段落或列表）
<code>![[image.png]] / ![[image.png 640]] / ![[image.png 640x360]]</code>	图片嵌入与尺寸
<code>![[audio.mp3]]</code>	原生 <code><audio></code> 控件；PDF 中渲染为标注文件名的占位卡片
<code>![[clip.mp4 480x270]]</code>	原生 <code><video></code> ；PDF 同上
<code>![[doc.pdf#page=2#height=400]]</code>	内嵌 PDF 查看器（页码/高度参数），附下载链接回退
<code>![[map.canvas]] / ![[table.base]]</code>	Canvas / Bases 打包为可访问的文件卡片

嵌入语义细节：被嵌入笔记的 frontmatter 剥离；脚注按「章 + 嵌入序号」命名空间化，多来源不冲突；**嵌入区段的标题不进入主目录与 PDF 书签**（降级为普通段落语义，锚点保留可链）；未列入结构的被嵌入笔记会被递归 `check`。

Properties（frontmatter）

YAML frontmatter 按 `markdown.obsidian.properties` 展示。类型保真：

- 文本、数字原样；布尔 → 复选框；日期字符串原样；
- 列表（`aliases` / `tags` / `cssclasses` 及任意列表值）→ 独立 value 胶囊；**逗号分隔的标量**（`tags: a, b`）按 Obsidian 规则拆分；
- `tags` 值渲染为标签胶囊（属性表内**不带 #**，与 Obsidian Properties UI 一致）；
- 值内 `"[[Note]]"` → 可点内部链接；URL → 外链；
- `cssclasses` 附加到本章 `<section>`；`aliases` 参与链接解析；
- 空 frontmatter（`---` 紧跟 `---`）合法且被吞掉；坏 YAML：check 报错、build 警告降级。

标签

`#tag`、`#嵌套/子标签`、`#带_下划线-连字符`、Unicode 与 emoji 均可；纯数字（如 `#1984`）不是标签。识别边界与 **Obsidian** 一致：`#` 必须位于行首或空白之后——紧贴标点（含全角 `:`、`,`）不识别。正文标签渲染为带 `#` 的胶囊，并带 `data-tag` 供 CSS 定制。

注释

`%%行内注释%%` 与多行 `%%` 块从输出中移除（HTML 与 PDF 均不可见）；围栏代码与行内代码中的 `%%` 原样保留；`\%%` 转义可见。

Callouts

> `[!note]` 自定义标题（支持 `**Markdown**`）
 > 正文同样是完整 Markdown。

- 全部官方类型及别名（`note` / `abstract-summary-tldr` / `info` / `todo` / `tip-hint-important` / `success-check-done` / `question-help-faq` / `warning-caution-attention` / `failure-fail-missing` / `danger-error` / `bug` / `example` / `quote-cite`），未知类型按自定义类型渲染并带 `data-callout` 钩子；
- 折叠：`[!type]+` 默认展开、`[!type]-` 默认折叠（HTML 用 `<details>`，PDF 保留标题呈现静态折叠态）；
- 任意层级嵌套；标题省略时用类型名的 Title Case。

任务列表

`- [ ]` 未完成；`- [x]` 完成、`- [-]` 取消（勾选 + 删除线弱化）；任意自定义状态字符（`[?]`、`[!]`、`[>]`）在方框中直接展示该字符，语义不丢失；均带 `data-task` 供 CSS 定制；可与普通列表任意混排嵌套。

Mermaid

```
```mermaid
flowchart LR
 A["Start"] --> B["End"]
 class A,B internal-link;
```
```

构建时离线渲染为 SVG（运行时脚本随 `dist/assets/` 分发）；PDF 管线等待全部图表完成后再分页。flowchart 节点标注 `internal-link` class 时，节点文本按 vault 规则解析为文内链接（已有 `click` 声明的节点不覆盖）。

其余共享语法

标准方言的一切照常可用：GFM 表格与删除线、`==高亮==`、脚注（含行内脚注 `^[...]`，按章命名空间化）、KaTeX（行内/块级，`\tag{}` 编号）、构建期代码高亮、图片尺寸 `![alt|300x200](...)`、图注（独立成段图片的 title → `<figcaption>`）、`\pagebreak` / `\newpage` 手动分页、六级 ATX 标题稳定锚点。

静态边界（刻意不做的）

| 能力 | 处理方式 | 归类 |
|-----------------------|--------------|--------|
| <code>query</code> 围栏 | 保留源码展示，不执行搜索 | 核心插件 |
| Dataview 等插件语言 | 保留源码展示 | 社区插件 |
| Canvas / Bases 交互视图 | 文件卡片 + 打包源文件 | 独立文件格式 |
| 悬停预览、图谱、重命名联动、复选框点击等 | 不属于静态输出 | 应用行为 |

「全覆盖」的口径是 Obsidian 官方 **Obsidian Flavored Markdown** 扩展表 + 静态 properties / tags / Mermaid / 官方附件格式；应用级动态行为明确标注而不是模拟。

第四部分 · 外观与 PDF

主题 · 样式钩子 · 官方打印管线

08 · 主题、样式与封面

本章内容

| | |
|--------------------------------------|----|
| style —— 全局样式入口 | 29 |
| 内联 CSS 的组装顺序（公开契约） | 29 |
| 常用 CSS 变量与钩子 | 30 |
| themes —— 多主题输出 | 30 |
| labels —— 标签文本与自定义容器 | 31 |
| 封面与封底 | 31 |
| 网页端形态 | 32 |

style —— 全局样式入口

```
style:
  accent_color: "#5f58b6"      # 链接与强调色 (--hb-accent)
  content_width: "920px"      # 屏幕阅读最大宽度
  base_font_size: "16px"      # 屏幕字号
  print_font_size: "10.5pt"   # PDF / 打印字号
  fonts:
    body: '"Source Han Sans SC", sans-serif'
    heading: 'Georgia, "Source Han Serif SC", serif'
    code: 'ui-monospace, Menlo, monospace'
  custom_css: showcase.css     # 字符串或数组；最后内联，优先级最高
```

`custom_css` 路径先按项目目录解析，找不到再回退到包内（因此可以直接写 `templates/theme-clay.css` 引用内置主题皮肤）。

内联 CSS 的组装顺序（公开契约）

KaTeX CSS → print.css（包内基础） → book.yml 覆盖（:root 变量 + @page） → custom_css

后者胜。`book.yml` 的样式类键都会翻译成 `:root` CSS 变量，自定义 CSS 用同名变量即可精确覆盖。

常用 CSS 变量与钩子

| 变量 / 钩子 | 含义 |
|--|--------------------|
| <code>--hb-accent</code> | 强调色 |
| <code>--hb-content-width</code> / <code>--hb-base-font-size</code> / <code>--hb-print-font-size</code> | 版心与字号 |
| <code>--hb-font-body</code> / <code>--hb-font-heading</code> / <code>--hb-font-code</code> | 字体栈 |
| <code>--hb-page-margin-top/right/bottom/left</code> | 页边距镜像（封面/封底/隔页定位用） |
| <code>--hb-page-height</code> | 页高镜像（特殊页恰占一页的计算基准） |
| <code>--hb-cover-bg</code> / <code>--hb-cover-color</code> / <code>--hb-back-bg</code> / <code>--hb-back-color</code> | 封面/封底配色 |
| <code>.chapter</code> / <code>.insert</code> / <code>.hb-divider</code> / <code>.hb-blank</code> | 条目类型钩子 |
| <code>data-chapter</code> / <code>data-entry</code> / <code>data-layout</code> | 条目定位钩子 |
| <code>.callout[data-callout="..."]</code> 、 <code>.obsidian-tag[data-tag="..."]</code> 、 <code>.task-list-item[data-task="..."]</code> | 方言元素钩子 |

themes —— 多主题输出

```

themes:
  - name: light                # 进入文件名，须匹配 [A-Za-z0-9][A-Za-z0-9_-]*
    label: "Light"
    default: true              # 默认主题使用标准文件名；其余为 handout.<name>.*
  - name: dark
    label: "Dark"
    style:                      # 每主题可覆盖 style / cover / back_cover / pdf
      accent_color: "#eaeaea"
      custom_css: templates/theme-dark.css
    pdf:
      header_footer_style:
        color: "#9a9a9a"

```

- 每个主题产出一对 `html/pdf` 变体，`dist/index.html` 落地页自动列出全部入口；
- `pdf` 的嵌套键（`page_numbers` / `header` / `footer` / `header_footer_style`）做二层合并——主题只写其中一键时不丢基础配置；

- 包内置四套皮肤可直接 `custom_css` 引用：`theme-dark.css`、`theme-sepia.css`、`theme-clay.css`、`theme-academic.css`；
- 深色/纸感主题的页边距底色由官方管线统一改写（见第 09 章「页面基底色」）。

labels —— 标签文本与自定义容器

```
labels:
  note: "说明"           # 覆盖内置显示文本
  tip: "提示"
  warning: "注意"
  danger: "危险"
  theorem: "定理"
  definition: "定义"
  example: "示例"
  exercise: "练习"
  keypoint: "要点"      # 新键 = 注册自定义容器 ::: keypoint
```

- 内置告示块：`::: note / tip / warning / danger`，冒号后可写自定义标题；
- 学术环境：`::: theorem / definition / example / exercise 名称`——不自动编号，编号写在名称里（如 `::: theorem 3.1 柯西不等式`）；
- `labels` 的新键注册为自定义容器（`tip` 样式基底 + `admonition-<key>` class 供 CSS 定制）；`pagebreak` 为保留字。

封面与封底

```
cover:
  enabled: true           # 默认 true
  background: "linear-gradient(145deg, #f4f0e7, #e8edf7)" # 颜色或渐变
  color: "#25304a"
  html: front/cover.html # 可选：自定义组件片段（占位符已填充）

back_cover:
  enabled: true           # 默认 false
  background: "linear-gradient(145deg, #25304a, #4a4d86)"
  color: "#ffffff"
  html: front/back.html
```

- 默认组件展示 `title / subtitle / authors / date / version`；自定义 `html:` 片段与模板同级受信；
- 封面永远满版出血（首页零边距）；封底在官方 PDF 中由「独立单页打印 + 整页覆盖」实现满版（浏览器直接打印时铺满正文区）；
- 主题可分别覆盖 `cover` / `back_cover`。

网页端形态

构建出的 HTML 自带屏幕工具条（返回 index、下载官方 PDF、打印按钮）、浏览器打印用的运行页眉（`<thead>` 跨页重复；官方管线自动隐藏之以免双页眉）与键盘 `Ctrl/Cmd+P` 拦截。这些均为 `screen-only`，不进入 PDF。

09 · 官方 PDF 管线

本章内容

[pdf.* 配置全键](#)

[页码语义：物理页 vs 逻辑页](#)

[目录页码回填](#)

[覆盖层机制（封面 / 封底 / 出血隔页 / running）](#)

[页面基底色](#)

[元数据与书签](#)

[浏览器打印 vs 官方 PDF](#)

[已知边界](#)

34

34

35

35

35

35

36

36

`mhb pdf` 用 Playwright Chromium 打开各主题 HTML、切换打印介质渲染，再做 PDF 后处理。本章解释全部 `pdf.*` 配置与管线机制。

pdf.* 配置全键

```
pdf:
  header_footer: true          # 页眉页脚总开关 (默认 true)
  toc_page_numbers: true      # 目录页码回填 (默认 true)
  cover_header_footer: false  # 封面是否也带页眉页脚 (默认 false)
  page_size: "A4"             # 传给 @page size
                              # (A3/A4/A5/B4/B5/letter/legal/ledger/tabloid 或显式尺寸)
  margin: "17mm 16mm 19mm 16mm"
  date_format: "YYYY.MM.DD"   # 仅页眉页脚的日期显示 (缺省用全局 date_format)
  page_numbers:
    format: "{{page}} / {{total}}" # 亦接受 x、x/x、page of total 等速记
    count_cover: false          # 封面不计页 (保留页面, 逻辑页码跳过)
    count_toc: false            # 目录不计页 (流内目录恒计页, 此键被忽略并警告)
    count_back_cover: false     # 封底不计页
  header:                      # 三槽位; 每章可被 running 策略覆盖
    left: "{{title}}"
    center: ""
    right: "{{version}}"
  footer:
    left: "Markdown Handout Builder"
    center: "{{page}} / {{total}}"
    right: "{{date}}"
  header_footer_style:
    font_size: "8px"
    color: "#667085"
    font_family: "..."       # 缺省用内置跨平台栈 (含 CJK)
    offset: "5mm"              # 垂直落点; 缺省按边距 38% 计算并钳制 3-8mm
```

页眉页脚由 Chromium 画在**页边距区** (CSS `@page` 边距盒不可用是引擎限制), 正文内容 100% 来自 HTML。

页码语义: 物理页 vs 逻辑页

`count_cover / count_toc / count_back_cover: false` 把对应区段**保留在 PDF 里但不计页**。实现: 管线打印两遍——「计页子集」(DOM 中移除被剔除区段, 让 Chromium 只对计页部分编号)与「全量无页眉版」, 再把被剔除区段的页面**拼接回**正确位置。因此:

- `{{page}} / {{total}}` 是逻辑页码;
- 被剔除区段的页面干净无页眉;
- 章节小目录始终在正文流内、恒计页; 流内主目录 (`contents:` 条目) 同样恒计页。

目录页码回填

两遍打印：第一遍读取 PDF 书签得到每个标题的真实页码，注入主目录与全部章节小目录的 `.toc-page` 占位符（同行追加、不改变分页），第二遍出正式件。数量对不齐时按标题文本贪心对齐，失败则告警放弃回填而不是给错页码。

覆盖层机制（封面 / 封底 / 出血隔页 / running）

- **封面**：首页零边距满版；计页且带页眉配置时，用无页眉版同页整页覆盖，保证背景干净；
- **封底**：正文流内是普通页（保证分页与页码稳定），另做**仅封底的独立单页打印**（此时它是首页，天然满版），后处理整页覆盖到末页；
- **出血隔页**（`divider.bleed: true`）：同封底机制。目标物理页靠隔页标题的书签目的地定位；定位失败降级为正文区背景并警告；
- **running 策略**：见第 05 章——按页区间遮罩或以 Chromium 渲染的条带替换页眉/页脚，正文与链接笔记不动。

全部覆盖层都发生在内容流之上，**书签、内链、目录页码一概保留**。

页面基底色

深色 / 纸感主题下，Chromium 打印的页边距底色与正文底色存在引擎级差异。官方管线解压每页内容流，把基底填充色改写为主题的打印底色（浅色主题则在流首插入整页底色），使页边距与正文浑然一体。Chromium 未来更改内容流格式时会**告警**而不是静默退化。

元数据与书签

- 元数据：Title（非默认主题附主题名）、Author(s)、Subject (subtitle)、Keywords、Creator、Producer、Language、创建/修改时间；
- 书签 (outline) 由全部标题生成：transclusion 副本的标题与 `navigation.outline: false` 的条目除外；
- `tagged: true` 生成带结构标签的 PDF（可访问性；书签与页码映射也依赖它）。

浏览器打印 vs 官方 PDF

| 维度 | 浏览器 Ctrl+P | 官方 mhb pdf |
|-----------|--------------------------------------|--------------------------|
| 页眉页脚 | <code><thead></code> 运行页眉（无页码） | Chromium 模板 + 每章策略，逻辑页码 |
| 封底/出血隔页 | 铺满正文区 | 满版出血 |
| 目录页码 | 无 | 真实页码回填 |
| 计页剔除 | 无 | <code>count_*</code> 全支持 |
| 元数据/书签后处理 | 无 | 全套 |

需要正式分发时，永远使用官方 PDF。

已知边界

- 混合纸张尺寸/横竖版（每区段独立 `@page`）刻意未做：Chromium 命名页在文档中段不可靠（会产生尾随空白页），未来需要分段渲染-合并管线；
- `break-before: recto/verso` 被 Chromium 忽略，未暴露为配置。

第五部分·参考

book.yml 全键速查与占位符总表

book.yml 全键参考

本章内容

[顶层元数据](#)

[结构与输出](#)

[条目通用键（chapter / insert / divider / part 可用）](#)

[各类型专有键](#)

[目录](#)

[pdf.*](#)

[style / cover / back_cover / themes / labels / markdown](#)

[占位符总表](#)

[CLI 速查](#)

38

39

39

40

40

41

42

43

43

类型标注：`str` 字符串、`num` 数字、`bool` 布尔、`list` 列表、`map` 映射。**加粗**为必填。本章页眉即 running 策略的活演示（`reference` layout：页眉中位 = 章名）。

顶层元数据

| 键 | 类型 | 默认 | 说明 |
|--------------|----------|------------|--|
| title | str | — | 书名；进封面、页眉、HTML <code><title></code> 、PDF 元数据 |
| subtitle | str | "" | 副题；进封面与 PDF Subject |
| language | str | zh-CN | HTML <code>lang</code> 与 PDF Language |
| date | str | 今天 | 原始日期（ <code>{{rawDate}}</code> ） |
| date_format | str | YYYY-MM-DD | 日期显示格式（ISO 预设或字面模式） |
| version | str/num | "" | 版次（如 <code>2.0-dialect</code> ）；进封面/封底与 <code>{{version}}</code> |
| authors | str/list | [] | 作者； <code>{{authors}}</code> 全列 / <code>{{author}}</code> 第一位 |
| keywords | list | [] | PDF Keywords |

结构与输出

| 键 | 类型 | 默认 | 说明 |
|-------------|----------|--------------------------------|---|
| chapters | list/str | — | 紧凑文档流；或指向 <code>.yml</code> 清单文件。与 <code>structure</code> 二选一 |
| structure | list/str | — | 语义文档流（ <code>type/part/include/layouts</code> 全能力） |
| layouts | map | <code>{}</code> | 命名条目默认值包；键为 layout 名 |
| output.html | str | <code>dist/handout.html</code> | HTML 输出（相对 <code>book.yml</code> ） |
| output.pdf | str | <code>dist/handout.pdf</code> | PDF 输出；非默认主题自动插入 <code>.<name></code> |

条目通用键（`chapter / insert / divider / part` 可用）

| 键 | 类型 | 默认 | 说明 |
|--------------------|-----------|-----------|--|
| class | str | "" | 附加 CSS class（空格分隔多枚） |
| layout | str | — | 引用 layouts 名 |
| chapter_toc | bool | 全局默认 | 开章小目录（仅章节） |
| toc | bool/str | true | <code>navigation.toc</code> / <code>label</code> 简写 |
| navigation.toc | bool | true | 进主目录 |
| navigation.label | str | 标题 | 主目录行文案 |
| navigation.level | int 1–6 | 1（部内自动+1） | 主目录层级 |
| navigation.outline | bool | true | 进 PDF 书签；false 须与 <code>toc:false</code> 同用 |
| flow.break_before | page/auto | page | 条目前分页 |
| flow.break_after | page/auto | auto | 条目后分页 |
| running | false/map | 继承全局 | 见第 05 章：header/footer 为 bool 或 left/center/right 槽位；style 为 font_size/color/font_family/offset |

各类型专有键

| 条目 | 专有键 |
|--------------------------|---|
| path
(chapter/insert) | path ; <code>as / role : chapter insert</code> (.html 不可作 chapter) |
| divider | title (bleed 时必填)、subtitle、note、background、color、bleed |
| blank | <code>blank: true 1-20</code> ; 长形另可 count/class/layout/flow |
| contents | 仅 <code>contents: true / type: contents</code> ; 全书至多一次 |
| part | divider 全键 + children (chapters/structure 同义) + defaults
(class/layout/chapter_toc/toc/navigation/flow/running) |
| include | include : 相对所在 YAML 的 <code>.yml</code> 路径 (长形 <code>type: include + path</code>) |
| layouts.<name> | extends + 条目通用键 (不含 path 类键) |

目录

| 键 | 类型 | 默认 | 说明 |
|---------------------|---------|-----------------|--------------|
| toc.enabled | bool | true | 生成主目录 |
| toc.title | str | 目录 | 主目录标题 |
| toc.depth | int 1-3 | 2 | 收录层级 |
| chapter_toc.default | bool | false | 每章默认开小目录 |
| chapter_toc.title | str | In this chapter | 小目录标题; "" 隐藏 |
| chapter_toc.depth | int 2-6 | 3 | 小目录收录 h2..hN |
| chapter_toc.class | str | "" | 附加 class |

pdf.*

| 键 | 类型 | 默认 | 说明 |
|-------------------------------------|------|------------------------------|---------------------------------|
| pdf.header_footer | bool | true | 页眉页脚总开关（每章 running 可 opt-in 回来） |
| pdf.toc_page_numbers | bool | true | 目录页码回填 |
| pdf.cover_header_footer | bool | false | 封面也带页眉页脚（要求封面计页） |
| pdf.page_size | str | A4 | 纸张 |
| pdf.margin | str | 18mm 16mm
20mm 16mm | 页边距（CSS 简写） |
| pdf.date_format | str | 全局值 | 页眉页脚日期格式 |
| pdf.page_numbers.format | str | {{page}} /
{{total}} | 页码格式（支持速记 x、x/x、page of total） |
| pdf.page_numbers.count_cover | bool | true | 封面计页 |
| pdf.page_numbers.count_toc | bool | true | 目录计页（流内目录恒计页） |
| pdf.page_numbers.count_back_cover | bool | true | 封底计页 |
| pdf.header.left/center/right | str | {{title}} / "" /
{{date}} | 页眉槽位 |
| pdf.footer.left/center/right | str | "" / 页码 / "" | 页脚槽位 |
| pdf.header_footer_style.font_size | str | 8.5px | 页眉页脚字号 |
| pdf.header_footer_style.color | str | #8a919a | 颜色 |
| pdf.header_footer_style.font_family | str | 内置栈 | 字体 |
| pdf.header_footer_style.offset | str | 按边距计算 | 垂直落点 |

style / cover / back_cover / themes / labels / markdown

| 键 | 类型 | 默认 | 说明 |
|-------------------------------|----------|----------|--|
| style.accent_color | str | #111111 | 强调色 |
| style.content_width | str | 860px | 屏幕版心 |
| style.base_font_size | str | 16px | 屏幕字号 |
| style.print_font_size | str | 11pt | 打印字号 |
| style.fonts.body/heading/code | str | 内置栈 | 字体栈 |
| style.custom_css | str/list | — | 自定义 CSS（项目路径优先，回退包内） |
| cover.enabled | bool | true | 封面开关 |
| cover.background / color | str | — | 封面配色（颜色或渐变） |
| cover.html | str | 默认组件 | 自定义封面片段 |
| back_cover.enabled | bool | false | 封底开关（其余键同 cover） |
| themes[] | list | 单默认主题 | name（ 必填 ，进文件名）、label、default、style、cover、back_cover、pdf（嵌套键二层合并） |
| labels.<key> | str | 英文默认 | 覆盖内置标签文本；新键注册自定义容器（ <code>pagebreak</code> 保留） |
| markdown.dialect | str | standard | standard obsidian |
| markdown.obsidian.vault_root | str | . | vault 根（相对 book.yml） |
| markdown.obsidian.properties | str | visible | visible hidden source |

占位符总表

| 占位符 | 可用范围 | 值 |
|--------------------------------------|------------------------------|---|
| {{title}} {{subtitle}} | 页眉页脚、封面组件、插页 | 书名 / 副题 |
| {{authors}} {{author}} | 同上 | 全体作者 / 第一作者 |
| {{date}} {{rawDate}} | 同上 | 格式化 / 原始日期 |
| {{version}} {{lang}} | 同上 | 版次 / 语言 |
| {{commit}} | 同上 | 笔记仓库短 hash（脏树带 <code>-dirty</code> ；非 git 为空） |
| {{theme}} | 页眉页脚 | 主题 label |
| {{page}} {{total}} | 页眉页脚、
page_numbers.format | 逻辑页码 / 总页数 |
| {{chapterTitle}}
{{sectionTitle}} | 每章 running 槽位 | 本章一级标题 |

未知占位符保留字面并在 `check` 警告。

CLI 速查

| 命令 | 备注 |
|------------------------------------|---------------------|
| mhb init [--force] | 脚手架 |
| mhb check | 全量校验，错误退出码 1 |
| mhb inspect [--json] | 展平结构表 |
| mhb build | HTML + index + 资源 |
| mhb pdf | 官方 PDF |
| mhb serve [--port N] | 预览 + 监听重建 |
| mhb all | check + build + pdf |
| mhb install-browser / install-deps | Chromium / CI 依赖 |
| 全命令 --config <file> | 指定配置 |

编辑器校验：`# yaml-language-server: $schema=../node_modules/markdown-handout-builder/book.schema.json`。

Markdown Handout Builder

Markdown Handout Builder

2026.07.11

2.0-dialect